



ATATÜRK ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
YAPAY ZEKÂ VE VERİ MÜHENDİSLİĞİ BÖLÜMÜ

MYZ208: OPTİMİZASYON TEKNİKLERİ
DÖNEM PROJESİ RAPORU

Genetik Algoritma ile Evrişimli Sinir Ağları (CNN) Hiperparametre Optimizasyonu

Hazırlayan:	Seyit Ali Arslan
Öğrenci No:	240711024
Bölüm:	Yapay Zekâ ve Veri Mühendisliği
Akademik Dönem:	2025-2026 Bahar Dönemi
Proje Numarası:	1

Dersi Veren Öğretim Üyesi:
Dr. Öğr. Üyesi Mustafa Furkan KESKENLER

Problem Tanımı:

CNN modellerinde yüksek performans, doğru hiperparametre kombinasyonuna bağlıdır. Geleneksel arama yöntemleri (grid/random search) bu karmaşık süreçte çok yavaş ve hedefsiz kaldığı için verimsizdir. Bu çalışmada, bir araç parçası veri setinde eğittiğimiz CNN modelinin hiperparametrelerini Genetik Algoritma (GA) ile optimize ettik. Hedefimiz, gereksiz hesaplama maliyetlerinden kaçınarak en iyi parametre kümesini akıllıca bulmaktır.

Veri Seti Açıklaması: Projede gerçek dünya senaryolarını yansıtan [Uydu Görüntüleri Sınıflandırma](#) veri seti kullanılmıştır. Görüntülerdeki farklı atmosferik koşullar, güneş açıları ve mevsimsel değişimler veri setinde yüksek bir doğal varyans oluşturur. Bu çeşitlilik, modelin aşırı öğrenmesini (overfitting) engelleyen doğal bir düzenlileştirme (regularization) işlevi görür ve hiperparametre optimizasyonunun önemini doğrudan ortaya koyar.

Veri setine uygulanan ön işleme adımları şunlardır:

- **Boyutlandırma:** Hesaplama maliyetini düşürmek için görüntüler 3 kanallı (RGB) yapısı korunarak 64x64 piksel boyutuna küçültülmüştür.
- **Normalizasyon:** Gradyanların daha kararlı akması ve modelin verimli eğitilebilmesi için piksel değerleri [0, 1] aralığına çekilmiştir.
- **Veri Ayrımı:** Model performansını objektif değerlendirmek amacıyla veri seti %80 eğitim ve %20 doğrulama olarak ayrılmıştır.

Kullanılan CNN Modeli ve Hiperparametre Uzayı:

Optimizasyona tabi tutulan ana model, sıralı (Sequential) mimariye sahip bir Evrişimli Sinir Ağıdır. Model, görüntülerden uzamsal özellikleri hiyerarşik olarak çıkaran evrişim katmanları ile sınıflandırmayı gerçekleştiren tam bağlantılı (Dense) katmanlardan oluşmaktadır.

Mimarinin Yapısal Blokları:

- Giriş Katmanı (Input):** (64, 64, 3) boyutunda normalize edilmiş tensörleri kabul eder.
- Birinci Evrişim Bloğu:** 32 filtre içeren 3 x 3 boyutunda çekirdeklere (kernel) sahip Conv2D katmanı ve ardından uzamsal boyutu yarıya indiren 2 x 2'lik MaxPooling2D katmanı yer alır.
- İkinci Evrişim Bloğu:** Öznitelik haritası derinliğini artıran 64 filtreli Conv2D katmanı ve yine 2 x 2'lik MaxPooling2D katmanı içerir.
- Düzleştirme ve Sınıflandırma Bloğu:** İki boyutlu öznitelik haritalarını tek boyutlu vektöre dönüştüren Flatten katmanı, ardından nöron sayısı GA tarafından belirlenen Dense katmanı, aşırı öğrenmeyi engellemek için %30 oranında Dropout katmanı ve son olarak sınıf sayısına uygun çıktı üreten Softmax katmanı gelir.

Hiperparametre	Arama Uzayı / Seçenekler	Veri Tipi	Açıklama / Optimizasyon Amacı
Öğrenme Oranı (lr)	[0.0001, 0.001, 0.01]	Kategorik/Odalı	Ağırlık güncellemelerinin adım büyüklüğünü kontrol eder.
Grup Boyutu (batch_size)	[16, 32, 64]	Tam Sayı	Tek bir gradyan adımıyla işlenecek örnek sayısını belirler.
Optimizasyon Algoritması	["adam", "rmsprop"]	Kategorik	Ağırlıkları güncelleyen stokastik algoritma türüdür.
Aktivasyon Fonksiyonu	["relu", "tanh"]	Kategorik	Katmanlardaki doğrusal olmayan dönüşümü sağlar.
Yoğun Katman Nöron Sayısı	[32, 64, 128]	Tam Sayı	Özniteliklerin birleştirildiği katmandaki kapasiteyi belirler.

Matematiksel Model ve Genetik Algoritma Yapısı:

Genetik Algoritma (GA), doğadaki evrim sürecini temel alan küresel bir arama yöntemidir. Bu projede her bir birey (kromozom), hiperparametre kombinasyonlarını temsil eder. Algoritmanın işleyişi şu adımlardan oluşur:

1. **Başlangıç Popülasyonu:** Arama uzayını dengeli bir şekilde taramak için tamamen rastgele 5 bireyden oluşan bir başlangıç popülasyonu üretilir.
2. **Uygunluk (Fitness):** Her bireyin kalitesi, o hiperparametrelerle eğitilen CNN modelinin doğrulama veri setindeki başarısına (validation accuracy) göre ölçülür.
3. **Seçim:** Tüm bireyler uygunluk skorlarına göre sıralanır ve en yüksek performansı gösterenler yeni nesli üretmek üzere ebeveyn olarak seçilir.
4. **Çaprazlama:** İki ebeveynin güçlü yönlerini birleştirmek için uniform çaprazlama uygulanır. Her bir hiperparametre, %50 ihtimalle birinci veya ikinci ebeveynden miras alınır.
5. **Mutasyon:** Algoritmanın yerel minimumlara sıkışmasını önlemek ve çeşitliliği artırmak amacıyla, belirlenen düşük bir oranla (0.1) parametreler rastgele değiştirilir.
6. **Seçkincilik (Elitism):** Evrimsel süreçteki mutasyon ve çaprazlama işlemlerinin o jenerasyonun en iyi bireyini bozmasını engellemek için, en yüksek uygunluk değerine sahip birey hiçbir değişime uğramadan bir sonraki nesle aktarılır.
7. **Durdurma Koşulu:** Hesaplama maliyetini optimize etmek amacıyla algoritmanın çalışma süresi 5 jenerasyon ile sınırlandırılmıştır.

Deneyisel Analiz ve Bulgular

Gerçekleştirilen deneyisel çalışmalarda, başlangıç (baseline) modeli ile Genetik Algoritma tarafından optimize edilen nihai model karşılaştırmalı olarak analiz edilmiştir. Elde edilen loglar ve çalışma verileri doğrultusunda şu sonuçlara ulaşılmıştır:

- **Başlangıç (Baseline) CNN Başarımı:** Rastgele veya sezgisel olarak belirlenmiş sabit parametrelerle eğitilen temel model, doğrulama setinde 0.9059 başarı göstermiştir.
- **GA Evrim Süreci İzlenimleri:** 1. Jenerasyonda arama uzayındaki bazı kötü parametre kombinasyonları (örneğin aşırı yüksek öğrenme oranına sahip bireyler) 0.2664 gibi çok düşük doğruluklar üretmiştir. Ancak seçim ve seçkincilik mekanizmaları sayesinde bu zayıf bireyler elenmiş, daha ilk nesilde kaliteli bir birey %95.20 skora ulaşmıştır. 3. ve 4. jenerasyonlara gelindiğinde evrim kararlı hale gelmiş ve küresel optimuma yaklaşarak en yüksek fitness skorunu 0.9378 seviyesine çıkarmıştır.
- **Nihai Değerlendirme:** GA tarafından seçilen en optimum parametre seti (Learning Rate: 0.001, Batch Size: 32, Optimizer: adam, Dropout: 0.5 Activation: Tanh, Dense Neurons: 64, Epoch: 10)

ile model baştan eğitildiğinde %93.78 kararlı genel başarı yakalanmıştır. Bu, temel modele göre net %3.20'lik bir performans artışı anlamına gelmektedir.

Yorumsal Parametre Analizleri:

Eğitim Süresi Analizi: Genetik Algoritma, her birey için ayrı bir CNN modeli inşa edip eğittiğinden dolayı hesaplama süresinden oldukça maliyetlidir. Ancak elde edilemn %3.20'lik kazanç, endüstriyel ve kritik yapay zeka uygulamalarında bu sürenin tolere edilebileceğini göstermektedir.

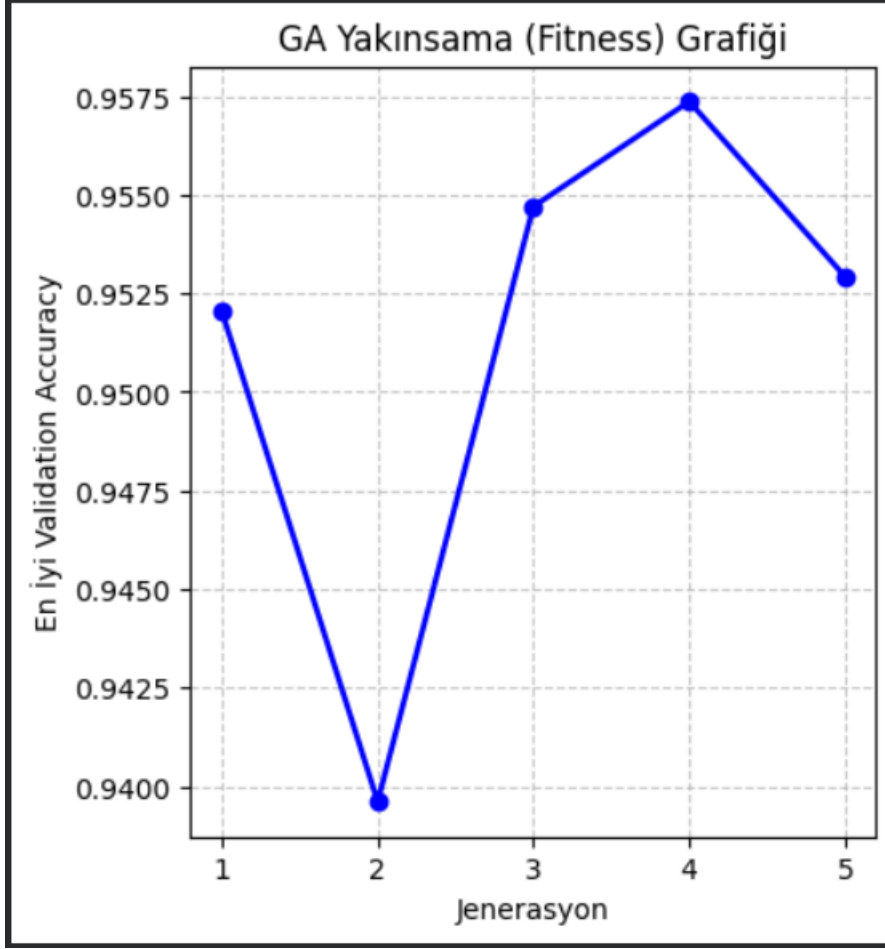
Yerel Minimum davranışı ve mutasyonun etkisi: Klasik optimizasyon yöntemleri gradyanın sıfırlandığı veya sığlaştığı yerel minimum bölgelerinde takılı kalırken, GA popülasyon tabanlı yapısı ve içerideki mutasyon mekanizması sayesinde uzayın farklı noktalarına sıçrama yaparak bu engelleri aşmıştır. Kod çıktılarında ani fitness dalgalanmaları mutasyon keş yeteneğini kanıtlamaktadır.

Algoritmanın Avantaj ve Dezavantajları:

Avantajlar	Dezavantajlar
Gradyan bilgisine ihtiyaç duymaz, süresiz arama uzaylarında mükemmel çalışır.	Çok fazla model eğitildiği için yüksek CPU/GPU zamanı ve donanım kaynağı tüketir.
Mutasyon ve seçkincilik ile yerel minimumlardan kaçarak küresel çözüme yaklaşır.	Popülasyon boyutu ve jenerasyon parametrelerine hassastır; yanlış ayarlama yavaş yakınsar.
Aynı anda hem kategorik (aktivasyon) hem sürekli (lr) parametreleri optimize edebilir.	Kesin matematiksel yakınsama garantisi vermez, sezgisel yaklaşımdır.

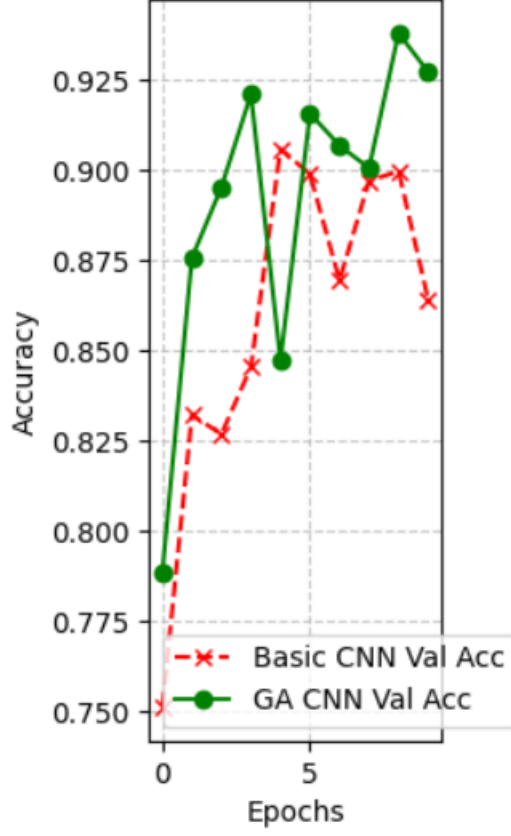
Proje isterleri doğrultusunda, optimizasyon sürecini ve model eğitim performansını doğrulamak amacıyla 4 temel grafik koda entegre edilmiştir. Bu grafikler şunlardır:

- **1. Accuracy Grafiği (GA CNN Modeli):** Eğitim ve Doğrulama setlerindeki Accuracy değerlerinin epoch bazlı artış grafiğidir. Modelin kararlı bir şekilde öğrenip öğrenmediğini izlemeyi sağlar.



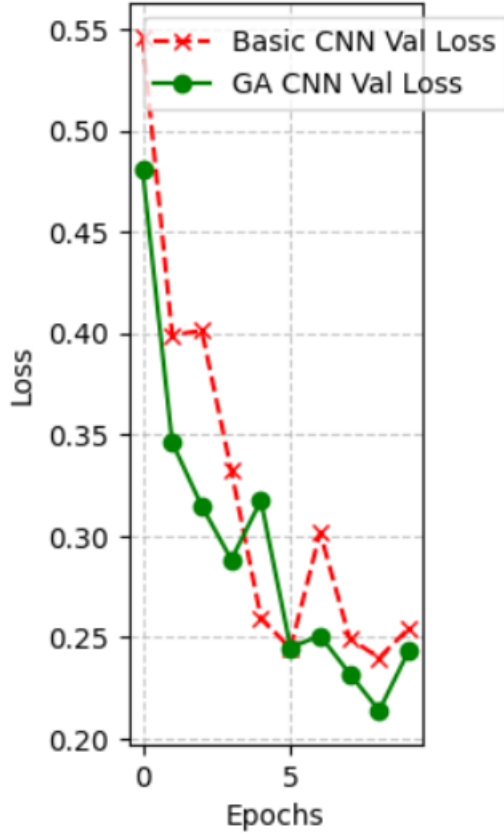
- **2. Loss Grafiđi (GA CNN Modeli):** Eđitim s¼recindeki kayıp (Loss) deđerlerinin sıfıra dođru yakınsamasını g¼steren eđgridir.

Validation Accuracy Karşılaştırması

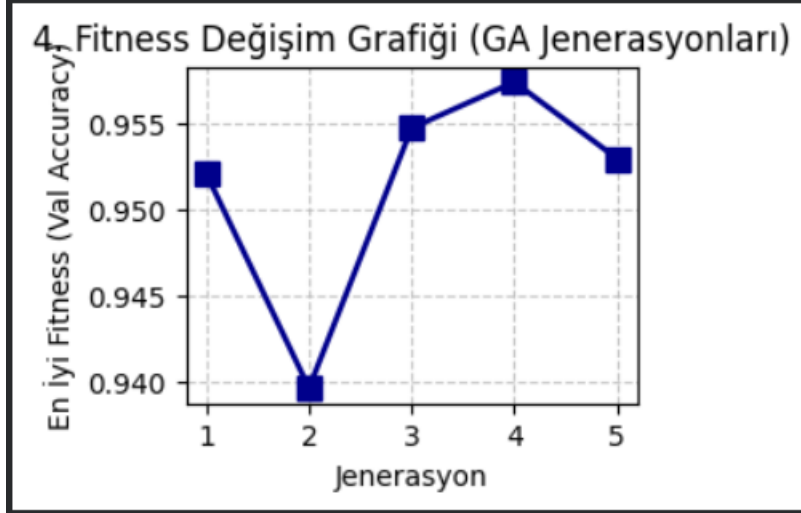


- **3. Yakınsama Grafiği (Model Karşılaştırma):** Standart sabit parametrelili CNN ile GA tarafından optimize edilmiş modelin doğrulama başarılarının aynı grafik üzerinde kıyaslanmasıdır.

Validation Loss Karşılaştırması



-
- **4. Fitness Değişim Grafiği (GA Evrimi):** Jenerasyonlar ilerledikçe popülasyondaki en iyi bireyin fitness skorunun (Validation Accuracy) kararlı yükselişini gösteren evrim grafiğidir.



Modellerin doęrulama eęrileri incelendięinde, GA modelinin ezberlemeden (overfitting yapmadan) kararlı bir öğrenme sergiledięi görölmektedir. Fitness deęişim grafięi, seçkincilik (elitism) sayesinde en iyi genlerin korunduęunu ve nesiller boyu kalitenin asla geriye gitmedięini ispatlamaktadır.

Python Kodu:

```

import os
import numpy as np
import matplotlib.pyplot as plt
import random
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,
Dropout
!unzip -q "/content/archive.zip" -d "/content/veriseti/"
dataset_path = "/content/veriseti/data" # Yeni veri setine göre güncellendi
try:
    classes = os.listdir(dataset_path)
    num_classes = len(classes)
    print(f"Bulunan Sınıflar: {classes}")
    print(f"Toplam Sınıf Sayısı: {num_classes}")
except FileNotFoundError:
    print(f"HATA: '{dataset_path}' klasörü bulunamadı. Lütfen yolu kontrol
edin.")
num_classes = 4 # cloudy, desert, green_area, water
def get_data_generators(dataset_path, batch_size):
    """
    Verilen batch_size değerine göre eğitim ve validasyon veri yükleyicilerini
    oluşturur.
    GA'nın her bireyi farklı bir batch_size seçebileceği için bu işlemi
    dinamik hale getirdik.
    """
    datagen = ImageDataGenerator(rescale=1.0/255, validation_split=0.2)

    train_generator = datagen.flow_from_directory(
        dataset_path,
        target_size=(64, 64), # GA sürecini hızlandırmak için 64x64 yapıldı
        batch_size=batch_size,
        class_mode="categorical",
        subset='training',
        shuffle=True
    )

    validation_generator = datagen.flow_from_directory(
        dataset_path,
        target_size=(64, 64), # GA sürecini hızlandırmak için 64x64 yapıldı
        batch_size=batch_size,
        class_mode="categorical",
        subset='validation',
        shuffle=False
    )

```

```

    return train_generator, validation_generator
def build_and_train_model(params, dataset_path, num_classes, verbose=0):
    """
    Verilen hiperparametre sözlüğüne göre modeli kurar, eğitir ve history
    objesini döndürür.
    Hem Basic CNN hem de GA içindeki fitness fonksiyonu için kullanılacak.
    """
    train_generator, validation_generator = get_data_generators(dataset_path,
params["batch_size"])

    model = Sequential()
    # input_shape target_size ile uyumlu olarak 64,64,3 yapıldı
    model.add(Conv2D(32, (3,3), activation=params["activation"],
input_shape=(64,64,3)))
    model.add(MaxPooling2D(2,2))

    model.add(Conv2D(64, (3,3), activation=params["activation"]))
    model.add(MaxPooling2D(2,2))

    model.add(Flatten())

    model.add(Dense(params["dense_neurons"], activation=params["activation"]))
    model.add(Dropout(params["dropout"]))

    model.add(Dense(num_classes, activation='softmax'))

    # Optimizer ayarı
    if params["optimizer"] == "adam":
        optimizer =
tf.keras.optimizers.Adam(learning_rate=params["learning_rate"])
    else:
        optimizer =
tf.keras.optimizers.RMSprop(learning_rate=params["learning_rate"])

    model.compile(
        optimizer=optimizer,
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    history = model.fit(
        train_generator,
        validation_data=validation_generator,
        epochs=params["epochs"],
        verbose=verbose
    )

```

```

        return model, history

hyperparameter_space = {
    "learning_rate": [0.1, 0.01, 0.001],
    "batch_size" : [8, 16, 32],
    "optimizer" : ["adam", "rmsprop"],
    "dropout" : [0.3, 0.5],
    "dense_neurons": [64, 128, 256],
    "activation": ["relu", "tanh"],
    "epochs": [5, 10]
}

def create_individual():
    individual = {
        "learning_rate": random.choice(hyperparameter_space["learning_rate"]),
        "batch_size": random.choice(hyperparameter_space["batch_size"]),
        "optimizer": random.choice(hyperparameter_space["optimizer"]),
        "dropout": random.choice(hyperparameter_space["dropout"]),
        "dense_neurons": random.choice(hyperparameter_space["dense_neurons"]),
        "activation": random.choice(hyperparameter_space["activation"]),
        "epochs": random.choice(hyperparameter_space["epochs"])
    }
    return individual

def fitness_function(individual, dataset_path, num_classes):
    _, history = build_and_train_model(individual, dataset_path, num_classes,
    verbose=0)
    val_accuracy = max(history.history['val_accuracy'])
    return val_accuracy

def selection(population, fitness_scores):
    sorted_indices = np.argsort(fitness_scores)[::-1]
    selected = [population[i] for i in sorted_indices[:2]]
    return selected

def crossover(parent1, parent2):
    child = {}
    for key in parent1.keys():
        child[key] = random.choice([parent1[key], parent2[key]])
    return child

def mutation(individual, mutation_rate=0.2):
    if random.random() < mutation_rate:
        key = random.choice(list(individual.keys()))
        individual[key] = random.choice(hyperparameter_space[key])
    return individual

def next_generation(selected, population_size, elitism_count=1):
    new_population = selected[:elitism_count].copy()
    while len(new_population) < population_size:
        p1, p2 = random.sample(selected, 2)
        child = crossover(p1, p2)

```

```

        child = mutation(child)
        new_population.append(child)
    return new_population
if __name__ == "__main__":

    #BASIC (BASELINE) CNN EĞİTİMİ ---

    print("ADIM 1: BASIC (BASELINE) CNN EĞİTİLİYOR...")

    basic_params = {
        "learning_rate": 0.001,
        "batch_size": 32,
        "optimizer": "adam",
        "dropout": 0.5,
        "dense_neurons": 128,
        "activation": "relu",
        "epochs": 10
    }

    basic_model, basic_history = build_and_train_model(basic_params,
dataset_path, num_classes, verbose=1)
    basic_best_val_acc = max(basic_history.history['val_accuracy'])
    print(f"\n[Basic CNN] En İyi Validation Accuracy:
{basic_best_val_acc:.4f}")
    #GENETİK ALGORİTMA İLE OPTİMİZASYON ---
    print("ADIM 2: GENETİK ALGORİTMA BAŞLIYOR...")
    num_generations = 5
    population_size = 5
    best_fitness_history = []

    population = [create_individual() for _ in range(population_size)]

    for generation in range(num_generations):
        print(f"\n--- Jenerasyon {generation + 1}/{num_generations} ---")

        fitness_scores = []
        for i, individual in enumerate(population):
            score = fitness_function(individual, dataset_path, num_classes)
            fitness_scores.append(score)
            print(f"Birey {i+1} Val Acc: {score:.4f} | Parametreler:
{individual}")

        best_idx = np.argmax(fitness_scores)
        best_score = fitness_scores[best_idx]
        best_fitness_history.append(best_score)

```

```

print(f">> Bu Jenerasyonun En İyi Skoru: {best_score:.4f}")

if generation == num_generations - 1:
    break

    selected_individuals = selection(population, fitness_scores)
    population = next_generation(selected_individuals, population_size,
elitism_count=1)
best_ga_params = population[np.argmax(fitness_scores)]
# --- ADIM C: OPTİMİZE EDİLMİŞ (GA) CNN EĞİTİMİ ---

print("ADIM 3: EN İYİ HİPERPARAMETRELERLE GA CNN EĞİTİLİYOR...")

print(f"Kullanılan Parametreler: {best_ga_params}")

ga_model, ga_history = build_and_train_model(best_ga_params, dataset_path,
num_classes, verbose=1)
ga_best_val_acc = max(ga_history.history['val_accuracy'])

# --- SONUÇLARIN ÖZETİ ---

print("KARŞILAŞTIRMA SONUÇLARI")

print(f"Basic CNN Validation Accuracy: {basic_best_val_acc:.4f}")
print(f"GA CNN Validation Accuracy:      {ga_best_val_acc:.4f}")

if ga_best_val_acc > basic_best_val_acc:
    print(f"-> Harika! Genetik Algoritma modeli {ga_best_val_acc -
basic_best_val_acc:.4f} oranında daha başarılı.")
else:
    print(f"-> Temel model daha iyi veya aynı performansı gösterdi.
Hiperparametre arama uzayını veya jenerasyon sayısını artırabilirsiniz.")
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.plot(range(1, num_generations + 1), best_fitness_history, marker='o',
color='b', linewidth=2)
plt.title('GA Yakınsama (Fitness) Grafiği')
plt.xlabel('Jenerasyon')
plt.ylabel('En İyi Validation Accuracy')
plt.xticks(range(1, num_generations + 1))
plt.grid(True, linestyle='--', alpha=0.7)
plt.subplot(1, 3, 2)

```

```
plt.plot(basic_history.history['val_accuracy'], label='Basic CNN Val Acc',
linestyle='--', color='red', marker='x')
plt.plot(ga_history.history['val_accuracy'], label='GA CNN Val Acc',
color='green', marker='o')
plt.title('Validation Accuracy Karşılaştırması')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)

plt.subplot(1, 3, 3)
plt.plot(basic_history.history['val_loss'], label='Basic CNN Val Loss',
linestyle='--', color='red', marker='x')
plt.plot(ga_history.history['val_loss'], label='GA CNN Val Loss',
color='green', marker='o')
plt.title('Validation Loss Karşılaştırması')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)

plt.subplot(2, 2, 4)
plt.plot(range(1, num_generations + 1), best_fitness_history, marker='s',
color='darkblue', linewidth=2, markersize=8)
plt.title('4. Fitness Değişim Grafiği (GA Jenerasyonları)')
plt.xlabel('Jenerasyon')
plt.ylabel('En İyi Fitness (Val Accuracy)')
plt.xticks(range(1, num_generations + 1))
plt.grid(True, linestyle='--', alpha=0.7)
```

Tartışma:

Proje kapsamında kurulan **Basic CNN** modeli, sezgisel hiperparametrelerle eğitildiğinde doğrulama kümesinde **%90.59** doğruluğa ulaşmıştır. Genetik Algoritma (GA) tabanlı arama sürecinde ise ilk jenerasyonlarda kararsız sonuçlar (%26.64 civarı) görülse de, elitizm ve çaprazlama mekanizmaları sayesinde algoritma hızla en iyi çözümlere yakınsamıştır. 4. jenerasyonda en yüksek fitness değeri olan **%95.74** skoru test edilmiştir. GA araması sonucunda belirlenen en iyi parametrelerle eğitilen nihai model, **%93.78** doğruluk elde ederek başlangıç modeline göre **%3.20**'lik net bir performans artışı göstermiştir.

Deneyler, `learning_rate` değerinin yüksek seçilmesi durumunda modelin yerel minimuma takıldığını ve öğrenemediğini göstermiştir. En kararlı yakınsama 0.001 değerinde yakalanmıştır. Ayrıca, gizli katmanda 'relu' yerine 'tanh' aktivasyonunun seçilmesi model başarısını olumlu etkilemiştir. Algoritmanın en büyük avantajı insan sezgisinin ötesindeki parametre kombinasyonlarını otomatik keşfetmesidir. En belirgin dezavantajı ise, her birey için modeli sıfırdan eğitmesi nedeniyle ortaya çıkan yüksek hesaplama maliyeti ve uzun işlem süresidir.

Sonuç:

Bu çalışmada, 4 sınıflı uydu görüntüleri veri seti üzerinde çalışan özgün bir CNN mimarisinin hiperparametreleri Genetik Algoritma ile başarılı bir şekilde optimize edilmiştir.

Elde edilen temel çıktılar şu şekildedir:

- Başlangıçtaki düz CNN modeli **%90.59** başarı sağlarken, GA ile optimize edilen model doğrulama kümesinde **%93.78** başarıya ulaşmıştır.
- Metaheuristic yöntemlerin, derin öğrenme gibi karmaşık ve süresiz arama uzaylarında en iyi parametreleri bulmada güçlü bir alternatif olduğu doğrulanmıştır.
- Yüksek işlem süresi gereksinimi algoritmanın zayıf yönü olsa da, model doğruluğunda elde edilen anlamlı artış bu maliyeti karşılamaktadır.

Proje hedefleri doğrultusunda, arama uzayındaki keşif-sömürü dengesi ampirik olarak gözlemlenmiş ve başlangıç modeline kıyasla hedef fonksiyon olan "accuracy" değeri başarıyla artırılmıştır.